For example, imagine you want to add a simple health-check endpoint to your existing app. This endpoint is a simple URL you can call that indicates whether your app is running correctly. You could easily create a health-check middleware using the `Run` extension, as you saw in listing 31.1, but then that's all your app can do. You want the health-check to respond only to a specific URL, `/ping`. Your Razor Pages should handle all other requests as normal.

TIP The health-check scenario is a simple example for demonstrating the `Map` method, but ASP.NET Core includes built-in support for health-check endpoints, which integrate into the endpoint routing system. You should use these instead of creating your own. You can learn more about creating health checks in Microsoft's "Health checks in ASP.NET Core" documentation: http://mng.bz/nMA2.

One solution would be to create a branch using the `Map` extension method and to place the health-check middleware on that branch, as shown in figure 31.1. Only those requests that match the `Map` pattern `/ping` will execute the branch; all other requests are handled by the standard routing middleware and Razor Pages on the main trunk instead, as shown in the following listing.

Listing 31.2 Using the `Map` extension to create branching middleware pipelines

```
app.UseStatusCodePages();                                    ❶

app.Map("/ping", (IApplicationBuilder branch) =>             ❷
{
    branch.UseExceptionHandler();                            ❸
    branch.Run(async (HttpContext context) =>                ❹
    {                                                        ❹
        context.Response.ContentType = "text/plain";         ❹
        await context.Response.WriteAsync("pong");           ❹
```